

Vyapar Margadarshan

Advanced Improvement Plan: Rule-Based Expert System, Explainable Anomaly Detection, Approval Intelligence, and Frontend Insights

Prepared for the post-defense improvement phase of the FYP. The plan is grounded in current rule-engine, expense workflow, Django/DRF, React dashboard, and explainable UI practices.

Area	Recommended direction
Main academic correction	Make the rule-based system visible through a separate knowledge base, rule engine, score explanation, and recommendation layer.
Backend priority	Refactor analytics/views.py so it becomes an API controller; move scoring into analytics/rule_engine.py and rules into analytics/rule_knowledge_base.py.
Frontend priority	Transform Insights.jsx into an explanation dashboard: risk score, triggered rules, contribution breakdown, reason, recommendation, and review action.
Advanced phase	Add DB-managed rules, rule audit history, approval policy rules, statistical baselines, duplicate detection, budget pressure, and rule performance metrics.

Date: July 3, 2026

1. Executive Summary

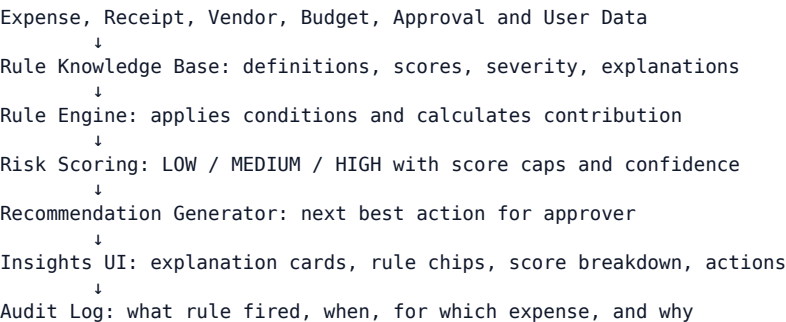
The defense feedback is valid: the project can contain rule-based logic and still fail to look like a rule-based expert system if rules are hidden as hardcoded if-statements inside a view function. The strongest improvement is not to add random features, but to expose the decision process as a structured pipeline: rule knowledge base, rule engine, risk scoring, recommendation generation, and frontend explanation.

Current best practices from Python rule-engine resources recommend separating business decision logic from application/controller code, so the logic can be maintained, tested, explained, and eventually configured without changing core views [S8][S9][S11][S21]. Expense management systems such as Odoo/Frappe-style workflows show that practical expense systems revolve around receipt capture, submission, approval states, role-based approval, and conditional workflow rules [S12][S13][S14][S15][S16].

For Vyapar Margadarshan, the best path is a staged refactor. First, extract the existing anomaly detection into a clear rule engine. Second, expand rule coverage. Third, show explanations in the frontend. Fourth, add tests and admin-visible rule management. Fifth, improve anomaly scoring with historical baselines and optional machine learning, while preserving the explainability of rules.

The core recommendation is: do not replace the project with a heavy commercial-style rules platform. Build a lightweight, project-specific rule engine first, because it is easier to defend, easier to test, and fits the existing Django + React architecture. Later, rule conditions can be moved to the database or a small expression engine.

Best final architecture



2. Research Findings Relevant to This Project

2.1 Rule engines and expert-system structure

A rule-based expert system is normally explained as a knowledge base plus an inference/rule engine. For an academic FYP, this distinction matters more than using a complex third-party library. The project should visibly store rule definitions away from controller code, then apply those definitions through a deterministic engine.

Practice	Meaning for Vyapar Margadarshan	Why it helps defense
Separate business logic from views	views.py fetches data and returns responses; rule_engine.py makes decisions.	The evaluator can see that decisions are not hidden inside API code.
Central rule metadata	Each rule has code, name, condition, score, severity, explanation, recommendation, owner, and version.	This becomes the knowledge base.
Explainable output	API returns triggered_rules with score contribution and reason.	The system can defend every risk score.
Configurable future	Rules can later move to DB/admin panel without rewriting the engine.	Shows maintainability and extensibility.

2.2 Lessons from similar expense and approval systems

Odoo's expense documentation highlights receipt upload, expense submission, dashboards, and approval processing [S12]. Odoo Studio approval rules show the idea of approval criteria before an action can proceed [S13]. Frappe/ERPNext discussions and plugins show expense approver tables, conditional workflows, and dynamic multi-level approval routing [S14][S15][S16].

The most useful lesson is not to copy their code. The useful design pattern is: expense systems become stronger when policy, approval routing, evidence/receipt compliance, role permissions, and audit history are first-class concepts.

Observed pattern	Implementation in Vyapar Margadarshan
Receipt evidence	Rules for missing receipt, unreadable receipt, receipt mismatch, duplicate receipt hash.
Conditional approval	Rules that decide whether manager, finance, or admin review is needed.
Workflow states	Submitted, pending review, needs clarification, approved, rejected, paid/closed.
Auditability	Store rule evaluations and approval comments for every decision.
Escalation	Rules for old pending expenses and high-risk expenses requiring faster or senior review.

2.3 Anomaly detection lesson

Open-source and research examples around invoice or financial anomaly detection commonly combine deterministic business rules with pattern-based or machine-learning methods. A Microsoft P2P anomaly detection example describes invoice extraction, retrieval of contract details, and anomalies based on predefined business rules [S18]. Finomaly presents a modular approach covering rule-based and ML-style financial anomaly detection [S19]. Research on legal invoice anomaly detection notes that traditional rule-based safeguards are helpful but may miss discrepancies, motivating ML or data-driven methods in later phases [S20].

For your project, this means the correct academic position is: rule-based scoring is the primary explainable method; statistical or ML anomaly detection is an advanced supplement, not a replacement.

3. Target Backend Architecture

3.1 Recommended folder structure

```
backend/analytics/  
  views.py           # API endpoints only  
  serializers.py     # optional API response serializers  
  rule_knowledge_base.py # static rules for phase 1  
  rule_engine.py     # applies expense-level rules  
  rule_context.py    # builds context: historical averages, vendor stats,  
budget_stats  
  rule_advisory.py   # business-level insights and recommendations  
  rule_audit.py      # stores or formats rule evaluation history  
  services/  
    anomaly_service.py # orchestrates anomaly detection workflow  
    budget_analysis_service.py  
    duplicate_detection_service.py  
  tests.py           # unit and API tests
```

3.2 Responsibility separation

File	Responsibility	Should NOT do
views.py	Authenticate, parse request, call services, return Response.	Should not contain scoring weights or long rule logic.
rule_knowledge_base.py	Define rule metadata and initial thresholds.	Should not query database.

File	Responsibility	Should NOT do
rule_context.py	Prepare facts needed by the rule engine: averages, vendor age, duplicate candidates, budget use.	Should not decide final risk.
rule_engine.py	Evaluate rules and calculate risk score, level, explanation, and recommendations.	Should not format frontend components.
rule_advisory.py	Create business-level recommendations from aggregated data.	Should not evaluate per-expense risk.
tests.py	Prove rules and API responses work.	Should not only test happy path.

3.3 Why not use a heavy third-party engine immediately?

Libraries such as business-rules, rule-engine, durable_rules, and Zato Rule Engine are useful references [S8][S9][S10][S11]. However, adding them immediately may create more explanation overhead than value for a final-year project. Your immediate need is a clear, defendable, testable, domain-specific rule system.

Option	When to use	Recommendation
Custom lightweight rule engine	Rules are simple if-condition checks over expense context.	Best for current project and defense.
DB-managed rules	Admin should enable/disable or edit thresholds without deployment.	Phase 2 advanced improvement.
Expression engine	Need dynamic conditions like amount > category_avg * 2.	Use later with strict validation.
durable_rules/Zato	Need event-driven, real-time, multi-source inference.	Too heavy for immediate FYP unless scoped carefully.

4. Rule Knowledge Base Design

4.1 Minimum rule schema

```
EXPENSE_REVIEW_RULES = {
    "MISSING_RECEIPT": {
        "name": "Missing Receipt Rule",
        "category": "COMPLIANCE",
        "description": "Expense has no attached receipt.",
        "score": 20,
        "severity": "MEDIUM",
        "recommendation": "Ask the submitter to attach a receipt before approval.",
        "enabled": True,
        "version": "1.0",
    },
    "HIGH_AMOUNT": {
        "name": "High Amount Rule",
        "category": "FINANCIAL_RISK",
        "description": "Expense amount is high compared with normal spending.",
        "score": 25,
        "severity": "HIGH",
        "recommendation": "Verify amount, business purpose, and approval authority.",
        "enabled": True,
        "version": "1.0",
    },
}
```

4.2 Stronger rule metadata for advanced phase

Field	Purpose
code	Stable identifier used in API, tests, logs, and frontend.
name	Human-readable name for cards and defense explanation.
category	Compliance, budget, duplicate, vendor, approval, receipt, spending pattern.

Field	Purpose
condition_type	Static Python function, threshold rule, expression rule, or aggregate rule.
threshold	Editable value such as 10000 amount, 80 percent budget use, 7 pending days.
score	Contribution to risk score.
severity	LOW, MEDIUM, HIGH, CRITICAL.
recommendation	Actionable advice for the approver.
evidence_fields	Data points used to trigger the rule: amount, budget_percent, receipt_status.
enabled	Allows disabling noisy rules without deleting them.
version	Supports audit/history when rules change.

4.3 Recommended rule catalog

Rule	Trigger	Score	Severity	Frontend explanation
MISSING_RECEIPT	No receipt file attached	+20	Medium	Receipt is required for verification.
MISSING_VENDOR	Vendor empty or unknown	+10	Low	Vendor details are missing.
WEAK_DESCRIPTION	Description too short or generic	+10	Low	Business purpose is not clear.
HIGH_AMOUNT	Amount above absolute threshold	+20 to +25	High	Large expenses require extra verification.
CATEGORY_OUTLIER	Amount > category median * 2 or z-score high	+15	Medium	Unusual for this category.
DUPLICATE_CANDIDATE	Same/similar amount, vendor, date	+20	Medium	Possible duplicate entry.
BUDGET_EXCEEDED	Expense pushes budget > 100 percent	+25	High	Budget limit may be exceeded.
BUDGET_PRESSURE	Budget use already above 80 percent	+15	Medium	Category budget is under pressure.
NEW_VENDOR	Vendor first seen recently	+10	Low	New vendor requires basic validation.
VENDOR_CONCENTRATION	Vendor dominates category spend	+10	Medium	Spending concentration may indicate dependency or abuse.
OLD_PENDING	Pending more than configured days	+10	Low	Approval delay requires follow-up.
RECEIPT_AMOUNT_MISMATCH	OCR total differs from submitted amount	+25	High	Receipt and submitted amount disagree.

5. Rule Engine Implementation Path

5.1 Phase 1 implementation pattern

```
# analytics/rule_engine.py
from .rule_knowledge_base import EXPENSE_REVIEW_RULES
```

```

RISK_LEVELS = [
    (0, 30, "LOW"),
    (31, 65, "MEDIUM"),
    (66, 100, "HIGH"),
]

def risk_level(score: int) -> str:
    score = max(0, min(score, 100))
    for low, high, label in RISK_LEVELS:
        if low <= score <= high:
            return label
    return "HIGH"

def apply_rule(code: str, reasons: list, evidence: dict | None = None) -> int:
    rule = EXPENSE_REVIEW_RULES[code]
    if not rule.get("enabled", True):
        return 0
    item = {
        "code": code,
        "name": rule["name"],
        "category": rule.get("category"),
        "message": rule["description"],
        "score": rule["score"],
        "severity": rule["severity"],
        "recommendation": rule["recommendation"],
        "evidence": evidence or {},
    }
    reasons.append(item)
    return int(rule["score"])

def evaluate_expense(expense, context: dict) -> dict:
    reasons = []
    score = 0

    if not getattr(expense, "receipt", None):
        score += apply_rule("MISSING_RECEIPT", reasons)

    if not getattr(expense, "vendor", None):
        score += apply_rule("MISSING_VENDOR", reasons)

    if expense.amount and expense.amount >= context.get("high_amount_threshold", 10000):
        score += apply_rule("HIGH_AMOUNT", reasons, {"amount": str(expense.amount)})

    if context.get("duplicate_candidate"):
        score += apply_rule("DUPLICATE_CANDIDATE", reasons, context["duplicate_candidate"])

    if context.get("budget_after_percent", 0) > 100:
        score += apply_rule("BUDGET_EXCEEDED", reasons, {
            "budget_after_percent": context.get("budget_after_percent")
        })

    score = min(score, 100)
    return {
        "expense_id": expense.id,
        "risk_score": score,
        "risk_level": risk_level(score),
        "triggered_rules": reasons,
        "recommendations": [r["recommendation"] for r in reasons],
    }

```

5.2 Service wrapper to keep views.py clean

```

# analytics/services/anomaly_service.py
from analytics.rule_context import build_context_for_expense
from analytics.rule_engine import evaluate_expense

class AnomalyDetectionService:
    def __init__(self, organization):
        self.organization = organization

    def evaluate_pending_expenses(self, queryset):
        results = []
        for expense in queryset:
            context = build_context_for_expense(expense, self.organization)
            results.append(evaluate_expense(expense, context))
        return results

# analytics/views.py
@api_view(["GET"])
@permission_classes([IsAuthenticated])
def anomalies(request):

```

```

organization = request.user.organization
pending_expenses = Expense.objects.filter(
    organization=organization,
    status="PENDING"
).select_related("category", "vendor", "submitted_by")

service = AnomalyDetectionService(organization)
results = service.evaluate_pending_expenses(pending_expenses)
return Response({"results": results})

```

5.3 Rule evaluation response contract

```

{
  "expense_id": 101,
  "title": "Office Chair Purchase",
  "amount": "18500.00",
  "risk_score": 75,
  "risk_level": "HIGH",
  "summary": "High risk due to missing receipt, high amount, and budget pressure.",
  "triggered_rules": [
    {
      "code": "MISSING_RECEIPT",
      "name": "Missing Receipt Rule",
      "score": 20,
      "severity": "MEDIUM",
      "message": "Expense has no attached receipt.",
      "recommendation": "Ask the submitter to attach a receipt before approval.",
      "evidence": {"receipt": null}
    }
  ],
  "recommendations": [
    "Ask the submitter to attach a receipt before approval.",
    "Verify amount, business purpose, and approval authority."
  ]
}

```

6. Improving Anomaly Score Detection

6.1 What is weak in the current scoring

Manual weights are not wrong, but they look arbitrary if the project does not explain them. The improvement is to expose score contribution, severity, evidence, and reason. Each rule should say: what it checked, why it matters, how many points it added, and what the approver should do.

6.2 Better scoring model without ML

Layer	Method	Example
Absolute threshold	Known policy limits	Amount above 10000 needs review.
Relative threshold	Compare to organization/category history	Amount > 2x category median.
Duplicate similarity	Same vendor/date/amount window	Same amount and vendor within 3 days.
Budget pressure	Budget usage before and after expense	Expense pushes category from 82 percent to 104 percent.
Receipt compliance	Receipt availability and OCR checks	Receipt missing or OCR total mismatch.
Approval behavior	Pending days, rejection frequency	Pending more than 7 days or repeated rejection.

6.3 Risk score formula

```

base_score = sum(triggered_rule_scores)
cap_score = min(base_score, 100)

optional_adjustments:
- Reduce duplicate rule score when similarity is weak.
- Increase high amount score if amount is extreme compared with category median.
- Cap low-severity rules so many small issues do not create a false high risk.
- Store confidence separately from risk when historical data is insufficient.

```

Important: risk and confidence are different. Risk says how serious the expense looks. Confidence says how much evidence the system has. A new organization with little data may produce a high-risk rule result with medium confidence.

6.4 Statistical baselines for advanced phase

Technique	Use	Implementation difficulty
Median and IQR	Robust category/vendor baseline; less affected by outliers.	Low
Z-score	Amount far from average; useful when data is enough and roughly normal.	Low-Medium
Moving average	Monthly spending spike detection.	Medium
Isolation Forest	Detect unusual combinations of amount, category, vendor, day, user.	Medium-High
Hybrid rule + ML	Rules explain common risks; ML suggests additional anomalies.	High, use only as future work.

6.5 Duplicate detection approach

Duplicate candidate rule inputs:

- same organization
- expense date within +/- 3 days
- amount difference <= 1 percent or exact same rounded amount
- same vendor OR high text similarity in title/description
- same receipt hash if file exists

Output evidence:

```
{
  "matched_expense_id": 99,
  "amount_similarity": "100%",
  "date_gap_days": 1,
  "vendor_match": true,
  "title_similarity": "87%"
}
```

7. Rule-Based Advisory Improvements

Per-expense anomaly detection answers: should this expense be reviewed carefully? Rule-based advisory answers: what is happening in the business? Keep these separate so the system is not confusing.

Advisory area	Rule examples	Frontend card
Spending control	Category overuse, monthly spike, high average amount	Spending Spike Alert
Budget health	Budget > 80 percent, budget exceeded, repeated overspend	Budget Pressure
Vendor risk	New vendor, vendor concentration, repeated high vendor bills	Vendor Concentration
Receipt compliance	Missing receipt ratio > 20 percent	Receipt Compliance
Approval performance	Pending count high, average pending age high	Approval Bottleneck
Policy quality	Many weak descriptions or missing vendors	Data Quality Warning

Advisory output shape

```
{
  "advisory_type": "BUDGET_PRESSURE",
  "title": "Office Supplies budget is under pressure",
  "severity": "MEDIUM",
  "message": "Office Supplies has used 86% of the monthly budget.",
  "evidence": {
    "category": "Office Supplies",
```



```

    "used_percent": 86,
    "remaining_budget": 4200
  },
  "recommendation": "Review non-essential purchases before approving more expenses."
}

```

8. Approval Workflow Improvements

8.1 Recommended workflow states

DRAFT -> SUBMITTED -> PENDING_MANAGER_REVIEW -> PENDING_FINANCE_REVIEW
-> NEEDS_CLARIFICATION -> APPROVED -> REJECTED -> CLOSED

Do not overbuild the workflow immediately. First make the statuses visible and consistent. Then connect rules to approval routing.

8.2 Rule-driven approval routing

Condition	Routing decision
Risk level LOW and amount below threshold	Manager approval only.
Risk level MEDIUM	Manager approval with rule explanation.
Risk level HIGH	Manager + finance/admin review.
Missing receipt	Needs clarification before approval, unless admin override.
Budget exceeded	Finance review required.
New vendor + high amount	Finance/admin review required.

8.3 Approval audit log

```

ApprovalAuditLog
- id
- organization
- expense
- actor
- action: SUBMITTED / APPROVED / REJECTED / REQUESTED_CHANGES / OVERRIDE
- previous_status
- new_status
- comment
- rule_snapshot_json
- created_at

```

The rule_snapshot_json is important. It stores the rule result at the time of approval. Even if rules change later, you can still explain why a previous expense was flagged.

9. Frontend Insights Page Design

9.1 Main UI goal

The frontend should not only say High Risk. It should show why. Explainable UI sources emphasize making logic visible and understandable, and risk-score UX discussions warn against showing raw numbers without interpretation [S23][S24]. Dashboard design research also supports using structured patterns such as overview, details-on-demand, filters, and focused analytical cards [S7].

9.2 Recommended Insights.jsx layout

Insights Page

```

[Top KPI Row]
- High-risk pending expenses
- Missing receipt rate
- Budget pressure categories
- Average approval delay

```

[Filters]

- Risk level
- Category
- Vendor
- Date range
- Rule type

[Main Grid]

- Left: anomaly cards / table
- Right: selected expense explanation panel

[Business Advisory Section]

- Budget pressure cards
- Vendor concentration cards
- Receipt compliance cards
- Approval bottleneck cards

9.3 Anomaly card design

Element	Purpose
Risk badge	LOW/MEDIUM/HIGH label; do not rely only on color.
Score bar	Show score out of 100 with text label.
Triggered rule chips	MISSING_RECEIPT +20, HIGH_AMOUNT +25.
Evidence line	Amount, vendor, receipt, budget percent, date.
Recommendation	Clear next action for approver.
Actions	View expense, request receipt, approve, reject, add note.

9.4 Frontend component structure

```

frontend/src/pages/Insights.jsx
frontend/src/components/insights/
  RiskSummaryCards.jsx
  InsightFilters.jsx
  AnomalyCard.jsx
  RiskScoreBar.jsx
  TriggeredRuleList.jsx
  RuleContributionBreakdown.jsx
  RecommendationBox.jsx
  AdvisoryCard.jsx
  EmptyInsightsState.jsx
frontend/src/hooks/
  useAnomalies.js
  useRuleAdvisories.js
frontend/src/services/
  analyticsApi.js

```

9.5 React state and data handling

Use React component decomposition and clear state ownership. React guidance recommends breaking UI into components and lifting shared state to the closest common parent [S5][S6]. For server data, TanStack Query is a strong fit because it manages fetching, caching, retrying, refetching, and mutation workflows [S4].

```

// hooks/useAnomalies.js
import { useQuery } from '@tanstack/react-query';
import { analyticsApi } from '../services/analyticsApi';

export function useAnomalies(filters) {
  return useQuery({
    queryKey: ['analytics', 'anomalies', filters],
    queryFn: () => analyticsApi.getAnomalies(filters),
    staleTime: 60_000,
  });
}

```

9.6 UI rules for a professional design

- Never show only a raw score. Always pair it with LOW/MEDIUM/HIGH and an action.
- Use progressive disclosure: card summary first, detailed rule evidence on expand or side panel.

- Group triggered rules by category: Compliance, Budget, Duplicate, Vendor, Spending Pattern.
- Show score contribution as a breakdown so the evaluator can see exactly why 75/100 happened.
- Use empty, loading, and error states so the page looks production-ready.
- Use consistent badges and card spacing from your design system; avoid one-off components.

10. Backend API Design

10.1 Endpoints

Endpoint	Method	Purpose	Phase
/api/analytics/anomalies/	GET	Return per-expense risk results.	Phase 1
/api/analytics/advisories/	GET	Return business-level rule advisories.	Phase 1-2
/api/analytics/rules/	GET	List rule knowledge base for frontend/admin view.	Phase 2
/api/analytics/rules//	PATCH	Enable/disable or adjust threshold.	Phase 3
/api/expenses//rule-audit/	GET	View rule snapshots for expense.	Phase 3

10.2 Response consistency

Since the frontend should not break, keep the existing anomaly endpoint name and add new fields gradually. Use additive changes: risk_score, risk_level, triggered_rules, recommendations, evidence. Avoid removing old fields until the frontend is updated.

11. Database-Managed Rules: Advanced Phase

After the static knowledge base is stable, add a database model so rules can be viewed and eventually edited from an admin page. This strongly supports the evaluator's knowledge base feedback.

```
class BusinessRule(models.Model):
    code = models.CharField(max_length=80, unique=True)
    name = models.CharField(max_length=150)
    category = models.CharField(max_length=50)
    description = models.TextField()
    score = models.PositiveSmallIntegerField(default=10)
    severity = models.CharField(max_length=20)
    recommendation = models.TextField()
    condition_type = models.CharField(max_length=50, default="PYTHON")
    threshold = models.JSONField(default=dict, blank=True)
    enabled = models.BooleanField(default=True)
    version = models.CharField(max_length=20, default="1.0")
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

Rule management UI

UI section	What to show
Rule list	Rule name, code, category, severity, score, enabled status.
Rule detail drawer	Description, recommendation, threshold, version, last updated.
Audit/history	Who changed threshold or enabled status.
Safe editing	Allow only threshold and enabled/disabled first, not arbitrary code.

12. Testing Strategy

Django REST Framework includes helper classes for API request testing [S2]. Django and DRF projects should test the rule engine separately from the API endpoint. This makes bugs easier to find and makes the rule-based claims defensible.

12.1 Test levels

Test type	Example
Unit test rule function	Missing receipt triggers MISSING_RECEIPT and adds 20.
Unit test score level	Score 70 returns HIGH.
Context test	Budget usage calculation returns 86 percent.
Duplicate test	Same vendor, same amount, date gap 1 day triggers duplicate.
API test	GET anomalies returns triggered_rules array.
Regression test	Old API fields remain available after refactor.

12.2 Example tests

```
def test_missing_receipt_rule_adds_score(expense_factory):
    expense = expense_factory(receipt=None)
    result = evaluate_expense(expense, context={})
    codes = [rule["code"] for rule in result["triggered_rules"]]

    assert "MISSING_RECEIPT" in codes
    assert result["risk_score"] >= 20

def test_high_risk_level_when_score_above_65():
    assert risk_level(66) == "HIGH"
    assert risk_level(31) == "MEDIUM"
    assert risk_level(30) == "LOW"
```

13. Implementation Roadmap

13.1 Best path for your current project

Step	Task	Files	Result
1	Create static rule knowledge base	rule_knowledge_base.py	Visible rules with metadata.
2	Create rule engine	rule_engine.py	Scoring moved out of views.
3	Create context builder	rule_context.py	Historical, budget, vendor, duplicate facts prepared cleanly.
4	Update anomalies endpoint	views.py	Same API endpoint, better response.
5	Add tests	tests.py	Proof that rules work.
6	Update Insights UI	Insights.jsx + components	Evaluators can see triggered rules and explanation.
7	Add advisories	rule_advisory.py	Business-level insights beyond per-expense anomalies.
8	Add rule list endpoint	views.py / serializers.py	Knowledge base visible in frontend.
9	Add audit snapshots	rule_audit.py / model	Approvals become explainable historically.
10	Advanced statistical baselines	rule_context.py	Less arbitrary anomaly detection.

13.2 Priority levels

Priority	Do this	Why
Must do	Rule knowledge base, rule engine, explained response, Insights explanation UI, tests.	Directly fixes defense feedback.
Should do	More rule catalog, advisory cards, duplicate detection, budget pressure, receipt compliance.	Makes the system feel genuinely rule-based.
Could do	Rule management page, DB-managed rules, audit log, TanStack Query, side panel UX.	Makes it production-like and easier to maintain.
Future work	ML anomaly detection, OCR mismatch confidence, rule performance metrics.	Advanced improvement after stable rules.

14. Coding Agent Prompts

14.1 Backend refactor prompt

Refactor the analytics anomaly detection into a clear rule-based expert system.

Current logic is inside backend/analytics/views.py, especially detect_expense_anomalies().

Create:

- backend/analytics/rule_knowledge_base.py
- backend/analytics/rule_engine.py
- backend/analytics/rule_context.py if needed

Requirements:

1. Move scoring weights and rule metadata into rule_knowledge_base.py.
2. Move rule evaluation into rule_engine.py.
3. Keep views.py as an API controller only.
4. Keep the existing anomalies endpoint working.
5. Add triggered_rules with code, name, score, severity, message, recommendation, and evidence.
6. Cap risk_score at 100 and return risk_level LOW/MEDIUM/HIGH.
7. Do not make unnecessary frontend changes in this step.
8. Add tests for missing receipt, missing vendor, high amount, duplicate candidate, budget exceeded, and risk levels.

14.2 Frontend prompt

Update frontend/src/pages/Insights.jsx to display explainable rule-based anomaly results.

For each anomaly, show:

- expense title
- amount, category, vendor, date
- risk_score out of 100
- risk_level badge
- triggered_rules with score contribution
- evidence/reason for each rule
- recommendation box
- action buttons for view/request clarification/approve/reject if already supported

Use clean dashboard patterns: KPI row, filters, anomaly card/table, details panel, loading state, empty state, and error state.

Keep design consistent with existing components and avoid AI-generated messy layout.

14.3 Rule management prompt

Add a read-only Rule Knowledge Base page.

Backend:

- Add GET /api/analytics/rules/ that returns all rule metadata.

Frontend:

- Create frontend/src/pages/RuleKnowledgeBase.jsx.
- Display rule code, name, category, score, severity, enabled status, description, and recommendation.
- Add filters by category and severity.
- For now, do not allow editing rules unless backend support exists.

15. Frontend Visual Pattern Recommendations

15.1 Page hierarchy

For the evaluator, the strongest screen is not a beautiful chart alone. It is a transparent decision screen. Use the following hierarchy:

- Top: summary KPIs showing total pending, high risk count, missing receipt rate, and budget pressure.
- Middle: anomaly list with risk score and triggered rule chips.
- Right/drawer: full explanation with contribution breakdown and evidence.
- Bottom: advisory insights and trend charts.

15.2 Example card copy

Office Chair Purchase
Risk Score: 75 / 100 Level: HIGH

Triggered Rules:
+20 Missing Receipt Rule - No receipt attached.
+25 High Amount Rule - Amount Rs. 18,500 is above office supplies threshold.
+15 Budget Pressure Rule - Office Supplies budget is already 86% used.
+15 Weak Description Rule - Description does not clearly explain business purpose.

Recommendation:
Ask for receipt, confirm business purpose, and compare with the remaining office supplies budget before approval.

16. Security, Permissions, and Data Integrity

Concern	Recommendation
Rule edit safety	Only admins can edit rules. Start with read-only rule page.
Audit integrity	Store rule snapshots at approval time.
Organization isolation	Every query must filter by organization.
Receipt files	Validate file types, size, and ownership.
Frontend actions	Do not allow approve/reject buttons without backend permission checks.
Score trust	Display score explanation and limitations.

17. Metrics to Prove Improvement

Metric	Why it matters
Number of rules in knowledge base	Shows rule-based depth.
Percent of flagged expenses with explanations	Shows explainability coverage.
Missing receipt rate	Useful business compliance insight.
High-risk pending count	Shows actionable anomaly detection.
Average approval delay	Shows workflow performance.
False positive review notes	Future way to tune rule weights.
Budget pressure categories	Connects expense rules to business decision-making.

18. Defense Explanation After Improvement

Use this explanation:

Vyapar Margadarshan uses a rule-based expert system approach for expense review. The rules are stored in a separate knowledge base with rule code, score, severity, explanation, and recommendation. The rule engine applies these rules to pending expenses using expense, receipt, vendor, budget, and approval context. Instead of only returning a final anomaly score, the system returns triggered rules and score contribution. The frontend displays those triggered rules so the approver can understand why an expense is considered low, medium, or high risk.

Honest limitation to mention:

The current rule weights are manually assigned based on domain assumptions and project scope. This makes the system transparent and explainable, but the weights are not yet statistically validated using a large real-world financial dataset. Future work can validate weights using historical records, add feedback-based tuning, or combine the rules with machine-learning anomaly detection while preserving explanations.

19. Final Checklist

- Rule Knowledge Base exists and contains at least 10 clearly named rules.
- Rule Engine returns triggered_rules with score contribution and recommendations.
- views.py no longer contains long scoring logic.
- Insights page displays risk score, risk level, triggered rules, evidence, and recommendation.
- At least 6 backend tests prove important rules work.
- Duplicate, budget pressure, and receipt compliance rules are implemented or documented as next phase.
- Rule Knowledge Base page is added or planned as advanced improvement.
- Defense explanation honestly states manual-weight limitation and future ML/statistical validation.

20. References

[S1] Django REST Framework. DRF for API controllers, serializers, and testing helpers.
<https://www.django-rest-framework.org/>

[S2] DRF Testing. Use APIRequestFactory/APITestCase style testing for endpoints.
<https://www.django-rest-framework.org/api-guide/testing/>

[S3] Django. Django encourages reusable, pragmatic, secure web development.
<https://www.djangoproject.com/>

[S4] TanStack Query. Server-state caching, retries, mutations, refetching, and lifecycle management.
<https://tanstack.com/query/latest>

[S5] React - Thinking in React. Break UI into components and identify state ownership.
<https://react.dev/learn/thinking-in-react>

[S6] React - Sharing State. Lift shared state to the closest common parent.
<https://react.dev/learn/sharing-state-between-components>

[S7] Dashboard Design Patterns. Dashboard patterns help structure analytical, narrative, and embedded dashboards.
<https://arxiv.org/abs/2205.00757>

[S8] business-rules PyPI. Captures business logic outside code and processes rules on the backend.
<https://pypi.org/project/business-rules/>

[S9] rule-engine PyPI. Expression-based Python rules with datetimes, regex, type hints, and thread safety.
<https://pypi.org/project/rule-engine/>

[S10] durable_rules GitHub. Rules framework for real-time event coordination and inference.
<https://github.com/jruizgit/rules>

[S11] Zato Rule Engine docs. Separating business logic from application code improves clarity and adaptability.
<https://zato.io/en/docs/4.1/rule-engine/tutorial.html>

[S12] Odoo Expenses docs. Expense systems support receipt upload, submission, approval, and expense dashboards.
<https://www.odoo.com/documentation/19.0/applications/finance/expenses.html>

[S13] Odoo Approval Rules. Approval rules define criteria before actions can be performed.
https://www.odoo.com/documentation/19.0/applications/studio/approval_rules.html

[S14] Frappe Expense Claim. Expense claim workflow and submit/approval concepts.
<https://docs.frappe.io/hr/expense-claim>

[S15] ERPNext expense approval issue. Shows conditional workflow and approvers table ideas for leave/expense claims.
<https://github.com/frappe/erpnext/issues/13520>

[S16] Ladder Approve GitHub. Dynamic multi-level approvals for expense claims and leave applications.
<https://github.com/Beveren-Software-Inc/Ladder-Approve>

[S17] Open source OCR expense workflow. Example of dynamic approval workflows, automatic rule detection, and OCR receipts.
<https://github.com/Rajgohel0312/Odoo-x-Amalthea-Expense-Management>

[S18] P2P anomaly detection. Invoice anomaly detection using extraction, contract retrieval, and predefined business rules.
<https://github.com/MSFT-Innovation-Hub-India/p2p-anomaly-detection>

[S19] Finomaly GitHub. Modular financial transaction anomaly detection using rule-based and ML methods.
<https://github.com/Filidetan597/finomaly>

[S20] Legal invoice anomaly paper. Rule-based safeguards can miss discrepancies; ML can improve anomaly detection using domain knowledge.
<https://arxiv.org/abs/2012.14511>

[S21] Python Rule Engine article. Rule engines abstract dynamic decision logic from application code.
<https://djangostars.com/blog/python-rule-engine/>

[S22] Expense approval best practices. Multi-level role-based approval, compliance checks, notifications, and escalations.
<https://www.alaan.com/blog/modern-expense-management-guide>

[S23] Explainable AI UI design. Interfaces should make system logic visible and trustworthy.
<https://www.eleken.co/blog-posts/explainable-ai-ui-design-xai>

[S24] Risk score UX patterns. Avoid raw score dumps; interpret scores with meaning, context, and action.
<https://medium.com/%40vamsiparasar1992/ux-patterns-for-ai-confidence-scores-and-risk-alerts-e8624e34cfd9>